claude-wsl-ubuntu / 4f364eb7-a600-4a2c-9b37-e5a313b6db9d

Metadata

- Source: `claude-wsl-ubuntu`
- Kind: `claude`
- Source file: `\\wsl.localhost\Ubuntu\home\avidullu\.claude\projects\-home-avidullu-projects-khelsutra-guru\4f364eb7-a600-4a2c-9b37-e5a313b6db9d\subagents\workflows\wf_fa7a2077-3a9\agent-a2425ecc6dc062cbe.jsonl`
- SHA-256: `1807344aab0d99ecf4f48a7f077fa4f0ce99e50a479876505601c20fc888c037`
- Source modified: `2026-06-29T10:51:28+00:00`
- Imported at: `2026-07-05T16:48:30+00:00`
- project: `wf_fa7a2077-3a9`
- session_id: `4f364eb7-a600-4a2c-9b37-e5a313b6db9d`

Transcript

1. user (2026-06-29T10:50:03.872Z)

Read /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/docs/RALLY_DETECTION_FIXES_PLAN.md IN FULL. This is the source of truth for the P2b decision. Extract precisely: the §7 tracker state (every row, especially the P2b row and whether P1/P2a are done), and §8 Q2 which has the exact A/B runbook commands + the promote/reject decision rule. Quote exact command lines verbatim. Capture the headline metric (tol_f1), secondary metrics (|?end|, rally-end precision, paired sign-test, CI delta), the served_regress_eps value, the exact one-line change to promote (which flag in which file), the doc-update steps, and any concrete file paths (golden manifest, features dir, output) the doc references. If a section is absent, say so explicitly. Your returned object IS data, not prose for a human.

2. assistant (2026-06-29T10:50:05.676Z)

I'll read the plan document in full.

3. user (2026-06-29T10:50:06.803Z)

```
1       <!--
2       PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.
3       -->
4
5       # Refactor Rally Detection Edge Cases & Parser Stability
6
7       > **Status:** `IN PROGRESS` · **Owner:** `Agent` · **Created:** `2026-06-19` · **Last
updated:** `2026-06-29`
8       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived`
9       > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`.
10      > **Relation to existing docs:** Extends `RALLY_DETECTION_QUALITY_REPORT.md`.
11      >
12      > **2026-06-29 update:** the three findings were re-verified against current `master`
(post the
13      > 38-commit advance that touched both `rally_gate.py` and `tracknet_runner.py`) ? **all
three were
14      > still present and unfixed.** P0 (the two in-repo edge cases) is **implemented + unit-
tested in this
15      > same PR (#393)**: the `_spread` fix (A) is behaviour-neutral and lands
unconditionally; the
16      > `_trim_inactive_tail` density fix (B) is a behaviour change on sparse footage (R4 is
**default-ON**
17      > in prod) so it lands behind a **default-OFF** `inactivity_temporal_density` flag,
promoted only
18      > after a golden-set measurement (§8 Q2). P1 lives in the sibling `sports-data-
collector` repo, so it
```

> ships as a **separate PR there** (cannot be in #393).

---

0. TL;DR

The rally detection pipeline is largely robust, but a deep code review highlighted three minor edge cases/bugs: a potential divide-by-zero or zero-spread in `_spread` for tiny sequences, a density calculation vulnerability for sparse tracks in `_trim_inactive_tail`, and silent skipping of malformed rows in the CVAT parsing script. This plan outlines the fixes for these issues to increase stability.

1. Problem & goal

The current implementation of the candidate windowing handles normal inputs extremely well (including handling blobs), but could fail or behave erratically under specific conditions:

- **Zero-spread calculation**: `_spread` in `rally_gate.py` evaluates to `0.0` for arrays of length 2, which could cause a zero deadband if used on short clips.
- **Sparse trailing frames**: `_trim_inactive_tail` calculates frame density based on *visible* frames. A long gap followed by a single active frame gives 100% density, artificially inflating the rally end.
- **Silent failure on malformed CSV**: `load_canonical_csv` in `rally_cvat.py` silently ignores rows without a valid `rally_number`.

Goal: Fix these three issues to harden the rally detection and ingestion pipelines without changing the existing heuristic thresholds.

4. Design / architecture

The changes are entirely localized to the identified functions:

- `rally_gate.py` ? **done (#393)**: `_spread` now rounds the p95 index UP with `math.ceil()`, so a
  2-element (or any short) input falls back to the full min..max range instead of collapsing to
  `0.0`. On large arrays ceil-vs-truncate differ by at most one sample, so the robust p5/p95 trim is
  unchanged in practice.
- `tracknet_runner.py` ? **done (#393), flag-gated default-OFF**: `_trim_inactive_tail` gains an
  `temporal_density` flag. When ON, the density denominator is the trailing window's **temporal
  length** (`min(trail_frames, frames[i] - frames[0] + 1)`) instead of `win_count` (the count of
  active *samples*, which sparse tracking inflates). Because `group` holds only active frames, with
  dense tracking the two denominators are identical ? so flipping the flag is a **no-op on
  well-tracked clips** and changes only the sparse-tracking case. **Correction (2026-06-29):** R4 is
  default-ON in production (`window_inactivity_end = 1.5`, promoted at the R1 milestone ? an
  earlier draft of this doc wrongly called it default-OFF), so the fix is a *behaviour change* on
  sparse footage and ships behind the new `inactivity_temporal_density` flag (**default-OFF**, plumbed
  config ? `trajectory_hybrid` ? eval `WindowingPreset`), to be promoted only after a golden-set
  measurement at the SERVED operating point (§8 Q2).
- `rally_cvat.py` ? **separate PR** (sibling `sports-data-collector` repo, not this one): introduce a
  `logger.warning` before each `continue` in `load_canonical_csv` so skipped/malformed rows are
  surfaced during ingestion.

6. Honest limits ? what this does NOT do

This will NOT change the core AI validation (Phase 3) or the existing `velocity_thresh` values. It solely tightens up boundary calculation edge cases. It will not suddenly "fix" rallies that the tracking model missed entirely due to lighting or occlusion.

```
57
58      ## 7. Deliverables & progress tracker   ? **source of truth**
59      Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
60
61      | ID | Deliverable | Depends on | Gated? | Status | PR |
62      |----|-------------|-----------|--------|--------|----|
63      | P0a | Fix `_spread` zero-collapse (behaviour-neutral, unconditional) | ? | No | ? |
#394 |
64      | P0b | `_trim_inactive_tail` temporal density behind `inactivity_temporal_density` flag
(default-OFF) | ? | No | ? | #394 |
65      | P1 | Warn on rows silently skipped by `load_canonical_csv` (sibling `sports-data-
collector`) | ? | No | ? | sdc#50 |
66      | P2a | Expose the flag in the `rally_seg_eval` A/B CLI (`--[vs-]inactivity-temporal-
density`) so the promotion run is one command | P0b | No | ? | this PR |
67      | P2b | **Promote P0b**: golden-set A/B at SERVED windowing ? flip flag default-ON if no
regression | P2a | ? owner | ? | ? |
68
69      ## 8. Open questions ? owner / external
70      1. ~~Do we want `_trim_inactive_tail` to fall back to the existing behavior if there are
very few
71         frames, or strictly enforce a minimum temporal density?~~ **Resolved (#393):** strict
temporal
72         density. Because `group` carries only active frames, the temporal denominator equals
the
73         sample-count denominator under dense tracking, so this is behaviour-preserving for
well-tracked
74         clips and only corrects the sparse case ? no separate few-frames fallback needed.
75      2. **Owner promotion gate (P2b) ? pending.** P0b ships flag-OFF, so the merge is a
76         **zero-regression no-op in production**. The A/B is now a single command (P2a wired
the toggle
77         into `rally_seg_eval`):
78
79         ```
80         python -m backend.eval.rally_seg_eval \
81             --manifest <golden_manifest.json> --features-dir <golden_features_dir> \
82             --preset served --vs served --vs-inactivity-temporal-density
83         ```
84
85         (base = SERVED with the flag OFF, candidate = SERVED with it ON; diffs tolF1 at the
**SERVED**
86         operating point ? R4 is default-ON in prod, so this measures B's real end-boundary
effect on
87         sparse footage.) **Promote** by flipping `IndexingConfig.inactivity_temporal_density`
to `True`
88         (a one-line PR) **only if tolF1 does not regress**. This step is owner-only: it needs
the golden
89         corpus + GPU/WSL-extracted trajectory features, which don't exist offline. Note
finding B is a
90         **no-op under dense tracking**, so the A/B only shows signal on clips where the
shuttle tracker
91         actually drops out ? which is exactly why it can't be measured on synthetic/offline
fixtures.
92
93      ## 9. Definition of done
94      - [x] `_spread` bug fixed and tested (behaviour-neutral, unconditional).
95      - [x] `_trim_inactive_tail` density fix implemented behind `inactivity_temporal_density`
(default-OFF),
96            plumbed config ? segmenter ? eval preset, tested in both flag states.
97      - [ ] **P2 promotion**: golden-set A/B at SERVED windowing, then flip the flag default-
ON if no
98            regression. *(owner-gated, real video ? see §8 Q2)*
99      - [ ] Warning log added for malformed rows in `rally_cvat.py`. *(sibling-repo PR,
separate)*
100     - [x] Tests updated and passing *(offline suite; golden-corpus regression is owner-gated
```

```
? see Q2)*.
101
102     ## 10. References
103     **Internal code anchors `[verified]`:**
104     - `backend/pipeline/detectors/rally_gate.py`
105     - `backend/pipeline/detectors/tracknet_runner.py`
106     - `src/parsers/cvat/rally_cvat.py`
107     **Internal docs:**
108     - `docs/RALLY_DETECTION_CODE_REVIEW.md`
109
```

4. assistant (2026-06-29T10:50:10.363Z)

I have read the document in full. Let me verify the exact text of the referenced code anchors and the eval module path to confirm the file paths the doc references, since the doc mentions specific config flags and CLI modules.

5. user (2026-06-29T10:50:16.223Z)

```
./backend/eval/rally_seg_eval.py
./backend/pipeline/detectors/rally_gate.py
./backend/pipeline/detectors/tracknet_runner.py
---grep config flag---
backend/config/models.py:199:    inactivity_temporal_density: bool = False
backend/eval/windowing.py:77:    inactivity_temporal_density: bool = False
backend/eval/windowing.py:102:            "inactivity_temporal_density":
self.inactivity_temporal_density,
backend/eval/windowing.py:137:        inactivity_temporal_density=bool(
backend/eval/windowing.py:138:            idx.inactivity_temporal_density
backend/eval/windowing.py:192:        inactivity_temporal_density=bool(
backend/eval/windowing.py:193:            idx.get("inactivity_temporal_density",
SERVED.inactivity_temporal_density)
backend/eval/windowing.py:237:
inactivity_temporal_density=preset.inactivity_temporal_density,
backend/pipeline/detectors/tracknet_runner.py:329:        inactivity_temporal_density: bool =
False,
backend/pipeline/detectors/tracknet_runner.py:426:
temporal_density=inactivity_temporal_density,
backend/eval/rally_seg_eval.py:587:        temporal_density = args.inactivity_temporal_density
backend/eval/rally_seg_eval.py:603:            args.vs_inactivity_temporal_density or
args.inactivity_temporal_density
backend/eval/rally_seg_eval.py:612:        preset = replace(preset,
inactivity_temporal_density=True)
backend/pipeline/segmenters/trajectory_hybrid.py:250:        inactivity_temporal_density =
idx_cfg.inactivity_temporal_density
backend/pipeline/segmenters/trajectory_hybrid.py:305:
inactivity_temporal_density=inactivity_temporal_density,
tests/test_rally_seg_eval.py:59:        inactivity_temporal_density=False,
tests/test_rally_seg_eval.py:60:        vs_inactivity_temporal_density=False,
tests/test_rally_seg_eval.py:68:    assert SERVED.inactivity_temporal_density is False
tests/test_rally_seg_eval.py:70:    assert _build_preset(args,
"base").inactivity_temporal_density is False
tests/test_rally_seg_eval.py:71:    assert _build_preset(args,
"cand").inactivity_temporal_density is False
tests/test_rally_seg_eval.py:76:    args = _eval_args(vs_inactivity_temporal_density=True)
tests/test_rally_seg_eval.py:77:    assert _build_preset(args,
"base").inactivity_temporal_density is False
tests/test_rally_seg_eval.py:78:    assert _build_preset(args,
"cand").inactivity_temporal_density is True
tests/test_rally_seg_eval.py:80:    both = _eval_args(inactivity_temporal_density=True)
tests/test_rally_seg_eval.py:81:    assert _build_preset(both,
"base").inactivity_temporal_density is True
tests/test_rally_seg_eval.py:82:    assert _build_preset(both,
"cand").inactivity_temporal_density is True
tests/test_windowing.py:115:    assert W.SERVED.inactivity_temporal_density is False  # default
```

```
OFF
tests/test_windowing.py:116:    d = {"indexing": {"inactivity_temporal_density": True}}
tests/test_windowing.py:117:    assert W.served_windowing_from_config(d,
"wasb").inactivity_temporal_density is True
tests/test_windowing.py:133:        "inactivity_temporal_density": False,  # finding-B density
fix, default OFF
```

6. assistant (2026-06-29T10:50:20.431Z)

The flag's exact promote location is `backend/config/models.py:199`. Let me confirm the exact class name there and check the eval CLI for the served_regress_eps and metric details that the doc references, to ground the data I return.

7. user (2026-06-29T10:50:22.029Z)

```
190         window_inactivity_end: float = 1.5
191         inactivity_rally_thresh: float = 0.20
192         inactivity_min_density: float = 0.30
193         # R4 density DENOMINATOR fix (code-review finding B), default OFF until measured at
the SERVED
194         # operating point. The R4 trim's fast-fraction denominator is the COUNT of active
samples in the
195         # trailing window, which sparse tracking inflates (one fast sample after a gap ?
100% dense ?
196         # tail held open). When True, the denominator is the window's TEMPORAL length
instead. No-op
197         # under dense tracking (sample-count == temporal-span), so this can flip with zero
regression on
198         # well-tracked clips; flag-gated default-OFF since R4 itself is default-ON. See
QUALITY_ITERATIONS.
199         inactivity_temporal_density: bool = False
200         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
201         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
```

8. user (2026-06-29T10:50:27.778Z)

```
backend/eval/rally_seg_eval.py:13:diff two windowings with a paired sign-test + CI (the per-tier
``delta``).
backend/eval/rally_seg_eval.py:97:    tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-
production costs precision) + the
backend/eval/rally_seg_eval.py:113:    tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds,
gts, ts, te)
backend/eval/rally_seg_eval.py:140:        "tol_f1": tol_f1,
backend/eval/rally_seg_eval.py:160:    "tol_f1",
backend/eval/rally_seg_eval.py:358:    sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the
per-tier 'number'. Also reports ? on the
backend/eval/rally_seg_eval.py:378:        "tol_f1",
backend/eval/rally_seg_eval.py:517:    if rows and "tol_f1" in rows[0]:
backend/eval/rally_seg_eval.py:523:            "| video | tolF1 | tolP | tolR | |?start| |
|?end| | split | merge |",
backend/eval/rally_seg_eval.py:528:                f"| {r['name']} | {r['tol_f1']:.3f} |
{r['tol_precision']:.2f} | "
backend/eval/rally_seg_eval.py:534:            f"**R2 aggregate (n={a['n']}):** tolF1
{_ci(a['tol_f1'])} · "
backend/eval/rally_seg_eval.py:554:        "| stratum | court | n | tolF1 | tolP | tolR | tIoU
F1@0.5 | seg_ratio |",
backend/eval/rally_seg_eval.py:561:                f"| {dur_label} | {court} | {a['n']} |
{_ci(a['tol_f1'])} | "
backend/eval/rally_seg_eval.py:809:            st = dd["sign_test"]
```

9. assistant (2026-06-29T10:50:31.318Z)

The doc itself does not mention `served_regress_eps` by name; let me confirm whether that token appears anywhere in the repo (the task asks me to capture its value), and grab the exact A/B decision/sign-test logic lines around 587-620 and 800-830 in the eval CLI to ground the runbook.

10. user (2026-06-29T10:50:37.333Z)

```
=== served_regress_eps anywhere ===
backend/eval/promotion.py:16:    REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``,
and ? when its served CI is
backend/eval/promotion.py:126:    served_regress_eps: float = 0.0,
backend/eval/promotion.py:147:      served_regress_eps: served regression tolerance (0.0 = "no
served-mean drop allowed").
backend/eval/promotion.py:164:        served_a, served_b, eps=served_regress_eps, n_boot=n_boot,
seed=seed
backend/eval/promotion.py:172:            proposal_a, proposal_b, eps=served_regress_eps,
n_boot=n_boot, seed=seed
backend/eval/serve_contrast.py:112:    served_regress_eps: float = 0.0,
backend/eval/serve_contrast.py:133:        served_regress_eps=served_regress_eps,
backend/eval/serve_contrast.py:204:        served_regress_eps=args.served_regress_eps,
tests/test_promotion.py:111:def test_served_regress_eps_lets_noise_level_wobble_through():
=== eps / epsilon in eval CLI ===
backend/eval/rally_seg_eval.py:238:    # tail keeps a few in-court points, enough to hold a
window, so point-clearing alone is too leaky
backend/eval/rally_seg_eval.py:461:
f"no_merge_rate_regress={g['no_merge_rate_regress']})",
backend/eval/rally_seg_eval.py:466:    if g["merge_rate_regress"]:
backend/eval/rally_seg_eval.py:469:            for r in g["merge_rate_regress"]
backend/eval/rally_seg_eval.py:471:        lines.append(f"  ? merge_rate REGRESSED on: {names}")
backend/eval/rally_seg_eval.py:734:        help="R2 tolerance-match end window (s); default 1.5
(keeps the rally-end honest)",
```

11. assistant (2026-06-29T10:50:40.008Z)

The `served_regress_eps` token lives in `backend/eval/promotion.py` (default `0.0`) and `serve_contrast.py`, not in the plan doc itself. Let me confirm the default value and its meaning precisely.

12. user (2026-06-29T10:50:40.699Z)

```
120         *,
121         proposal_a: Optional[Sequence[float]] = None,
122         proposal_b: Optional[Sequence[float]] = None,
123         structural: bool = False,
124         min_n: int = 5,
125         p_threshold: float = 0.05,
126         served_regress_eps: float = 0.0,
127         n_boot: int = 2000,
128         seed: int = 0,
129     ) -> ServedGateResult:
130         """Promote a cue ONLY if it does not regress at the SERVED windowing (the
foundational rule).
131
132         The honest bar (``should_promote_for_user``: min-N floor + ?F1 CI excludes 0 + sign
test) is
133         applied to the **served** scores ? the operating point production runs ? so a
proposal-only
134         win can never carry a promotion. The optional ``proposal_*`` scores are scored too
and
135         reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
their
136         own).
137
138         Args:
139           served_a/served_b: per-video held-out scores for variant A (control) and B (the
```

```
cue),
140             measured at the **SERVED** windowing. These drive the decision.
141           proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
Reported for
142             contrast; the served regression guard still has the final say.
143           structural: forwarded to ``should_promote_for_user`` ? a structural guard (e.g.
Gate-A vs
144             held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
145             the served default if it regresses served. So for ``structural=True`` we report
146             ``keep_on`` honestly but ``promote`` reflects the served-safety check.
147           served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").
148
149         Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
```

13. assistant (2026-06-29T10:50:45.046Z)

I have everything I need. The plan doc itself does not name `served_regress_eps`, `|?end|`, rally-end precision, paired sign-test, or CI delta as part of the §8 Q2 decision rule ? those are eval-harness concepts that live in the code, not quoted in the doc. I'll report the doc's actual contents verbatim and flag what is absent in the doc explicitly, while grounding the code-side values I confirmed.

14. user (2026-06-29T10:51:28.786Z)

Structured output provided successfully